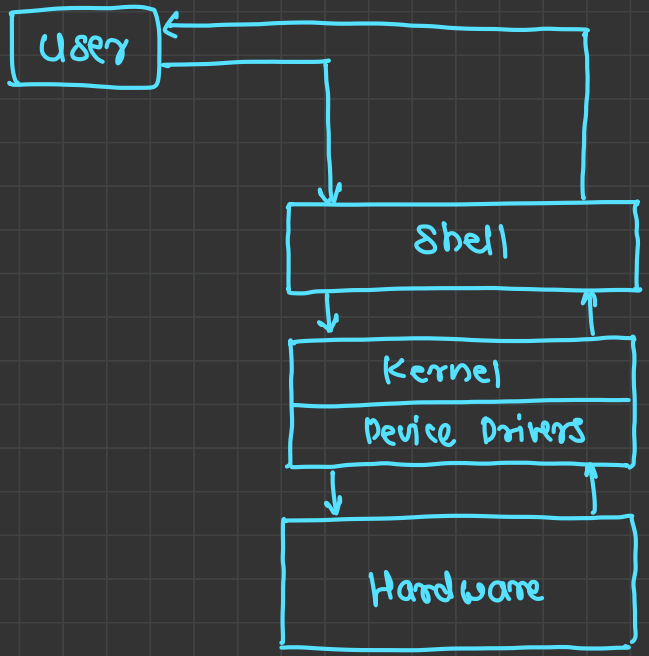
# Powershell




**Desktop manager**

Windows → Aero | metro
macos → Acqua user interface
linux → GNOME, Cinamon, xfce, KDE

Shell → parser | interpreter
understands user commands

**Windows** → powershell

**Linux** → sh → shell
bash → bourne again shell
csh → c shell
tsh → t shell

**macos** → zsh → z shell

# What is PowerShell?



- **A Task Automation and Configuration Management Framework**
  - Consists of a command-line shell and a scripting language
- **Built on the .NET Framework (.NET Core for cross-platform)**
  - This means it works with **objects**, not just text.
- **Cross-Platform**
  - Originally Windows-only, now available on **Windows, Linux, and macOS**
- **Two Main Components**
  - **The Shell:** An interactive command-line interface (CLI)
  - **The Scripting Language:** A full-fledged language for building complex automation.
-

# Why Automate with PowerShell?



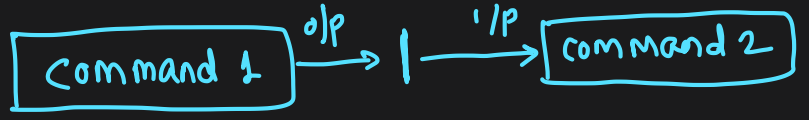
- **Efficiency:** Automate repetitive, time-consuming tasks (e.g., user onboarding, file cleanup).
- **Consistency:** Ensure tasks are performed the same way every time, reducing human error.
- **Scalability:** Manage one server or ten thousand servers with the same script.
- **Integration:** Seamlessly manage Windows, Azure, Microsoft 365, VMware, AWS, and more.
- **Power:** Access underlying system APIs (WMI, CIM, .NET) for deep system control.

# The Cmdlet



- A lightweight command used in the PowerShell environment
- **Naming Convention: Verb-Noun**
  - This makes commands predictable and discoverable
  - Examples:
    - Get-Process (Gets running processes)
    - Set-Service (Modifies a service)
    - New-Item (Creates a new file or directory)
- **Key Commands for Discovery**
  - Get-Command: Find all available cmdlets
  - Get-Help: Learn how to use a cmdlet. (e.g., Get-Help Get-Process -Examples)
  - Get-Member: See what an object can do. (e.g., Get-Process | Get-Member)

# The Pipeline




- **The Problem with Older Shells:** They pass text between commands
- **The PowerShell Way:** It passes full **.NET objects** through the pipeline
- **How it Works**
  - The output of one command becomes the input for the next
  - Command-A | Command-B | Command-C
- **Example**
  - Get-Process | Where-Object {$_.CPU -gt 100} | Sort-Object CPU -Descending
  - - Get-Process gets all process objects
  - - Where-Object filters those objects, keeping only those with CPU > 100
  - - Sort-Object sorts the filtered objects by CPU usage

# PowerShell Scripting Fundamentals

# Adjust the Execution Policy



- Get Execution Policy

```
Get-ExecutionPolicy
```

- Set Execution Policy

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

# Variable Declaration



- **Basic Declaration:** Prefix with $

```
$name = "Alice"
$age = 30
```

- **Case-Insensitive:** $Name and $name refer to the same variable.
- **Typed Declaration (Strong Typing)**

```
[string]$city = "Mumbai"
[int]$count = 5
```

- **Read-Only Variables**

```
Set-Variable -Name "PI" -Value 3.14 -Option ReadOnly
```

- **Constant**

```
New-Variable -Name "MAX" -Value 100 -Option Constant
```

- **Scope Modifiers:** $global:, $script:, $local:, $private:

```
$global:appName = "MyApp"
```

# Data Types

| Type | Example | Description |
|---|---|---|
| [string] | "Hello" | Text |
| [int] | 42 | 32-bit integer |
| [long] | 9999999999 | 64-bit integer |
| [double] | 3.14159 | Floating point |
| [decimal] | 10.99 | High-precision decimal |
| [bool] | $true, $false | Boolean |
| [datetime] | Get-Date | Date/time |
| [array] | @(1,2,3) | Collection |
| [hashtable] | @{a=1} | Key-value pairs |
| [pscustomobject] | [PSCustomObject]@{} | Custom object |

# Arrays



- **Literal Syntax**

```
$fruits = "Apple", "Banana", "Cherry"
$numbers = 1, 2, 3, 4, 5
```

- **Using @()**

```
$empty = @()
$mixed = @(1, "two", 3.0, $true)
```

- **Strongly Typed Array**

```
[int[]]$scores = 10, 20, 30
[string[]]$names = "Alice", "Bob"
```

- **Range Operator**

```
$range = 1..10                # 1,2,3,...,10
$alpha = 'a'..'e'             # a,b,c,d,e
```

- **From Cmdlet Output:**

```
$processes = Get-Process
```

# Array Operations



- **Access by Index (0-based)**

```
$fruits[0]          # Apple
$fruits[-1]         # Cherry (last item)
$fruits[0,2]        # Apple, Cherry
```

- **Slice a Range:**

```
$fruits[0..1]       # Apple, Banana
```

- **Count Items**

```
$fruits.Count       # 3
$fruits.Length      # 3
```

- **Add Items**

```
$fruits += "Date"                # Creates new array
$list = [System.Collections.ArrayList]@()
$list.Add("Mango")               # Efficient add
```

# Array Operations



- **Sort**

```
$sorted = $numbers | Sort-Object
$desc   = $numbers | Sort-Object -Descending
```

- **Filter**

```
$even = $numbers | Where-Object { $_ % 2 -eq 0 }
```

- **Select / Project**

```
$names = $users | Select-Object -Property Name, Email
```

- **Search**

```
$fruits -contains "Apple"       # True
$fruits -notcontains "Grape"    # True
$fruits -like "*an*"            # Banana
```

- **Iterate**

```
$numbers | ForEach-Object { $_ * 2 }
```

# Loops



- **For loop**

```
for ($i = 0; $i -lt 5; $i++) {
    Write-Host "Iteration $i"
}
```

- **Foreach**

```
$fruits = "Apple", "Banana", "Cherry"
foreach ($fruit in $fruits) {
    Write-Host "Fruit: $fruit"
}
```

- **ForEach-Object Cmdlet (pipeline-friendly)**

```
$fruits | ForEach-Object {
    Write-Host "Fruit: $_"
}
```

- **Key Difference**
  - foreach loads everything into memory first → faster for small sets.
  - ForEach-Object streams → better for large pipelines.

# Functions



- **Simple Function**

```powershell
function Say-Hello {
    Write-Host "Hello, World!"
}
Say-Hello
```

- **With Parameters**

```powershell
function Greet-User {
    param(
        [string]$Name,
        [int]$Age
    )
    Write-Host "Hello $Name, you are $Age years old."
}
Greet-User -Name "Alice" -Age 30
```

# Snippets

# System Information Reporter



```powershell
# Computer name and current user
Write-Host "Computer Name : $env:COMPUTERNAME"
Write-Host "Logged-in User: $env:USERNAME"
Write-Host ""

# Operating System details
$os = Get-CimInstance -ClassName Win32_OperatingSystem
Write-Host "OS Name       : $($os.Caption)"
Write-Host "OS Version    : $($os.Version)"
Write-Host "Last Boot Time: $($os.LastBootUpTime)"
Write-Host ""

# CPU details
$cpu = Get-CimInstance -ClassName Win32_Processor
Write-Host "CPU           : $($cpu.Name)"
Write-Host "Cores         : $($cpu.NumberOfCores)"
Write-Host ""

# Memory details (converted to GB)
$ramGB = [math]::Round($os.TotalVisibleMemorySize / 1MB, 2)
Write-Host "Total RAM (GB): $ramGB"
Write-Host ""
```

# Check Disk Space & Alert



```powershell
$threshold = 15  # Warn if free space is below 15%

Write-Host "Checking disk space..." -ForegroundColor Cyan
Write-Host ""

Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3" | ForEach-Object {
    $sizeGB = [math]::Round($_.Size / 1GB, 2)
    $freeGB = [math]::Round($_.FreeSpace / 1GB, 2)
    $percentFree = [math]::Round(($_.FreeSpace / $_.Size) * 100, 2)

    if ($percentFree -lt $threshold) {
        Write-Host "⚠ WARNING: Drive $($_.DeviceID) has only $percentFree% free ($freeGB GB of $sizeGB GB)" -ForegroundColor Red
    }
    else {
        Write-Host "✓ OK: Drive $($_.DeviceID) has $percentFree% free ($freeGB GB of $sizeGB GB)" -ForegroundColor Green
    }
}
```

# List Running Services



```powershell
param(
    [string]$Filter = "*"
)

Write-Host "Running services matching '$Filter':" -ForegroundColor Cyan
Write-Host ""

Get-Service -Name $Filter |
    Where-Object { $_.Status -eq "Running" } |
    Select-Object Name, DisplayName, Status |
    Format-Table -AutoSize
```

# Bulk Create AD Users from CSV



```powershell
Import-Module ActiveDirectory
$pwd = ConvertTo-SecureString "P@ssw0rd!" -AsPlainText -Force

Import-Csv C:\ad_users.csv | ForEach-Object {
    New-ADUser `
        -Name "$($_.FirstName) $($_.LastName)" `
        -SamAccountName $_.Username `
        -UserPrincipalName "$($_.Username)@contoso.com" `
        -GivenName $_.FirstName `
        -Surname $_.LastName `
        -Department $_.Department `
        -Path $_.OU `
        -AccountPassword $pwd `
        -Enabled $true
}
```

```
FirstName,LastName,Username,Department,OU
Alice,Smith,asmith,HR,"OU=HR,DC=contoso,DC=com"
Bob,Jones,bjones,IT,"OU=IT,DC=contoso,DC=com"
Charlie,Brown,cbrown,Finance,"OU=Finance,DC=contoso,DC=com"
```